

THE SIMPLEST ALGORITHM IN THE COURSE

KNN Classification and Regression

Look at the points nearest to the one you are asking about, and let them answer for it. That is the whole idea.

■ ONE ALGORITHM, TWO JOBS

The same steps solve both classification and regression.

01 · CLASSIFICATION

The neighbours vote

Output is a category. Count which category appears most among the neighbours.

02 · REGRESSION

The neighbours are averaged

Output is a number. Take the average of the neighbours' values.

Everything before that last step is the same in both cases. Classification first.

Three Steps, Start to Finish

01

Initialise the K value

How many neighbours should get a say. Any whole number greater than zero.

02

Find those K neighbours

Measure the distance from the test point to every training point, keep the K closest.

03

Count and take the majority

Whichever category owns most of those neighbours is the prediction.

Notice what is missing: there is no training. The model keeps the data as it is and does all the work at prediction time.

STEP 01

K Is Just How Many Neighbours You Ask

$$K > 0$$

$$K = 1, 2, 3, 4, 5, \dots$$

K equals three means three neighbours get a say. K equals five means five.

HYPERPARAMETER

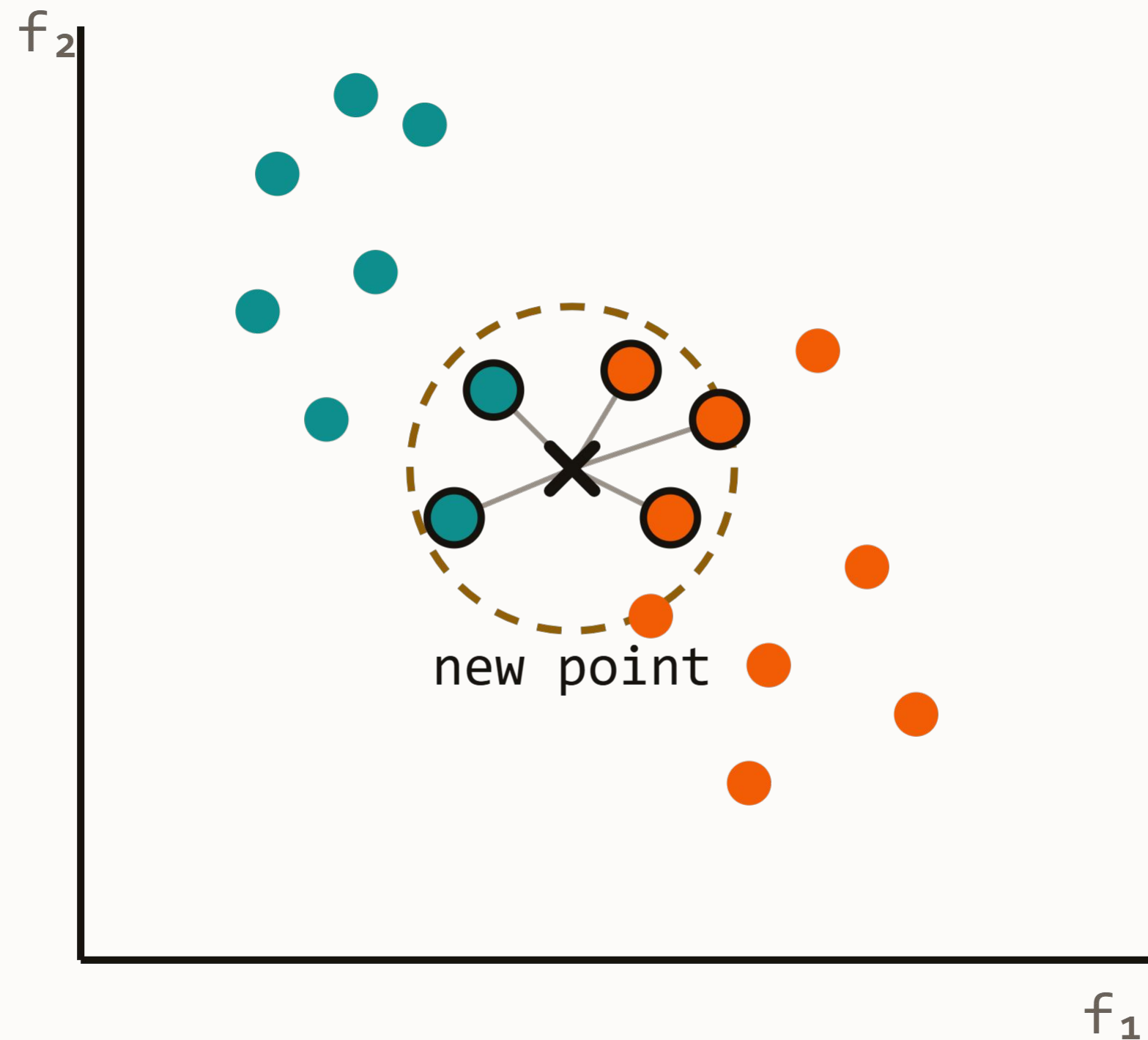
A number we choose ourselves, not one the model learns.

There is no K that is correct for every dataset. The right value depends on the data in front of you.

So we try a range of values, check the accuracy of each, and keep the winner. That search is what hyperparameter tuning means.

For the rest of this explanation, assume the tuning gave us $K = 5$.

Find Five, Then Count



2

neighbours in category 0

3

neighbours in category 1

Three beats two, so the new point is predicted as category 1.

The ring is not part of the model. It is only there to show which five points came out closest.

With three or five categories nothing changes. You count more piles and still take the biggest.

Why Not $K = 1$, and Why Odd Numbers?

K TOO SMALL

With K equal to one, a single stray point decides the whole answer.

If that nearest point happens to be a mistake in the data, your prediction is a mistake too.

K TOO LARGE

You start counting points that are nowhere near the test point, and the model drifts towards whichever category is most common overall.

KEEP K ODD FOR TWO CATEGORIES

$K = 4$

2 against 2, no majority

$K = 5$

3 against 2, always decided

An odd count cannot tie between two categories, so the vote always resolves.

The reliable way to pick K is still to try a range of values and compare accuracy. These two rules only narrow the range down.

■ THE MISSING PIECE

How does the computer decide what “nearest” means?

It needs a formula that takes two points and returns one number. Two are used with KNN.

01

Euclidean distance

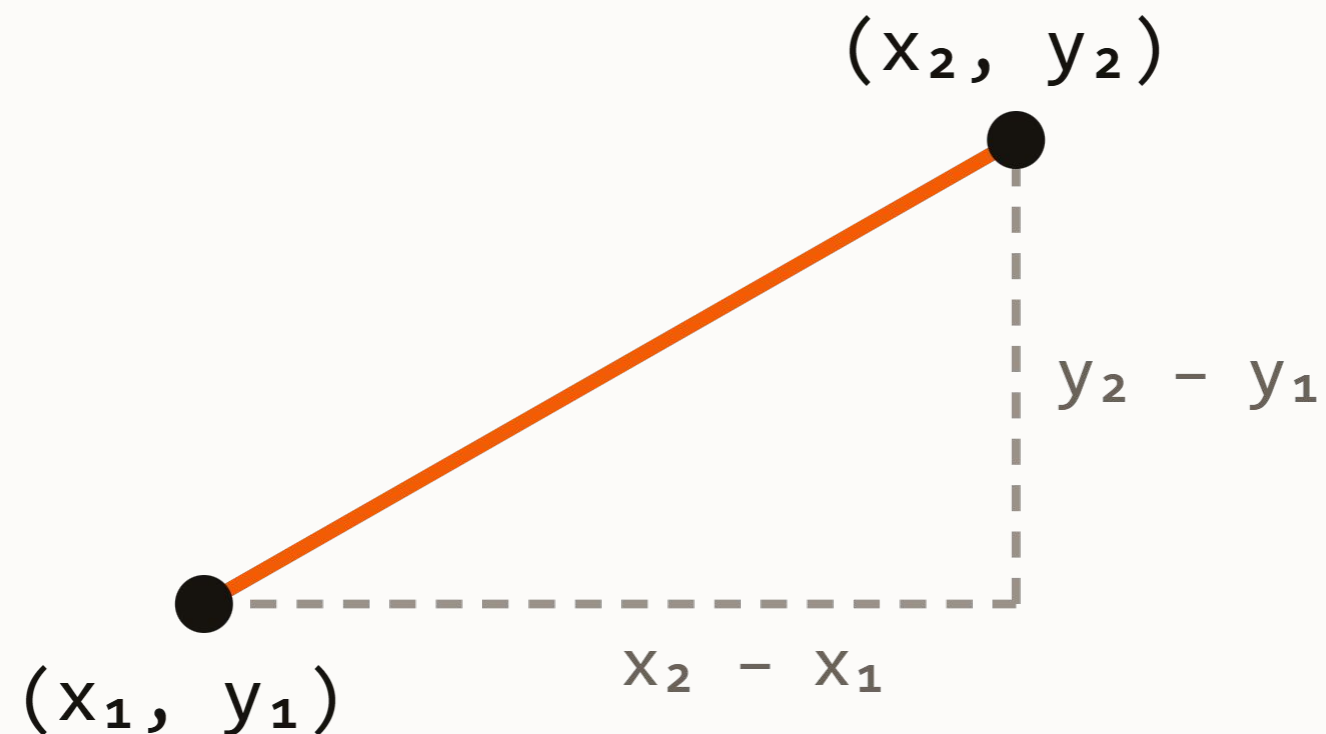
The straight line between the two points.

02

Manhattan distance

Along the sides, never diagonally.

Euclidean Distance



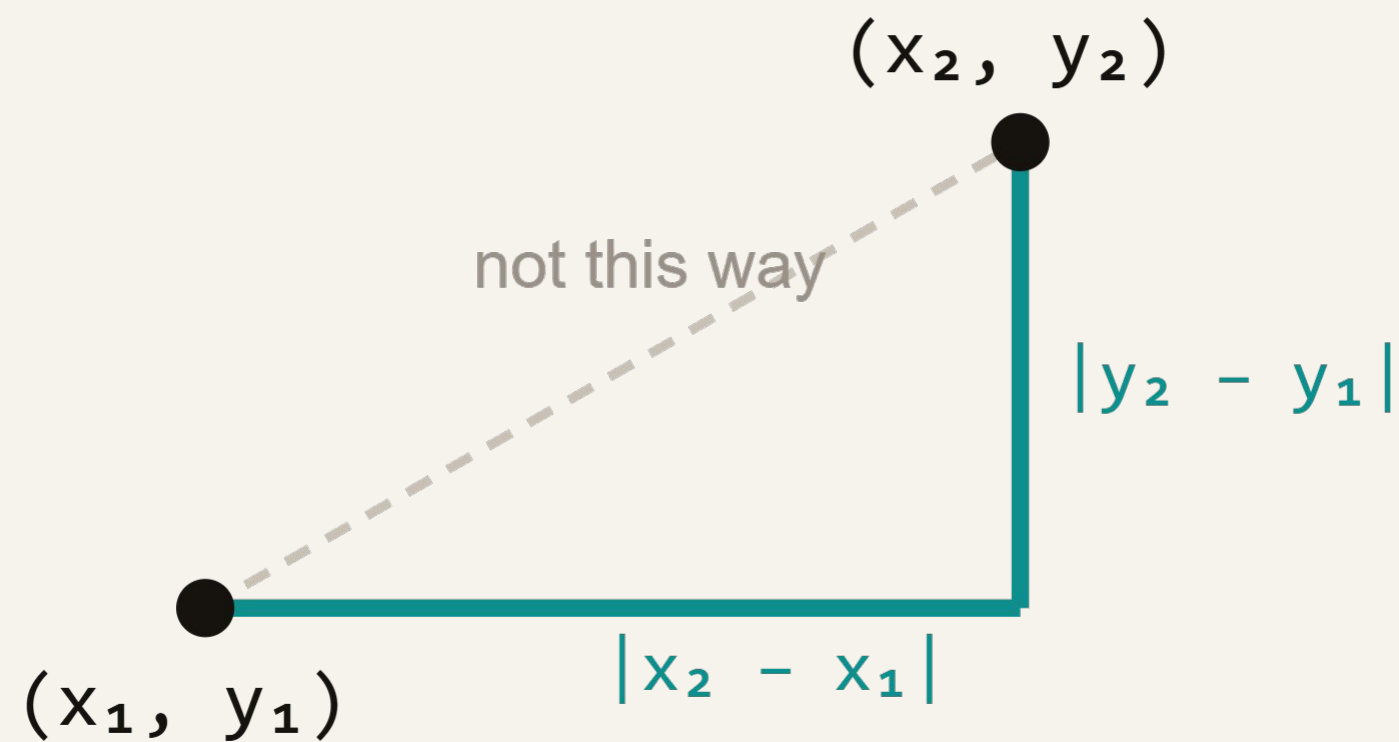
$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

This is Pythagoras. The straight line is the hypotenuse of the triangle made by the horizontal gap and the vertical gap.

Three features? Add a $(z_2 - z_1)^2$ term inside. Ten features, ten terms. The pattern never changes.

A flight from Bangalore to Delhi takes this distance. It goes straight, it does not follow the roads.

Manhattan Distance



$$d = |x_2 - x_1| + |y_2 - y_1|$$

No squares and no square root. Take the size of each gap and add them up.

The name comes from cities built in blocks. A cab drives along the roads, it cannot cut through the buildings.

Same two points, two different answers. Which one is right depends on whether travel is free in any direction or restricted to the grid.

Which One Should You Use?

Most projects start with Euclidean, and most of the time it is fine.

Treat it like K: try both, keep whichever scores higher. It is one more hyperparameter.

Manhattan tends to hold up better when you have a very large number of columns.

THIS MATTERS MORE THAN THE CHOICE

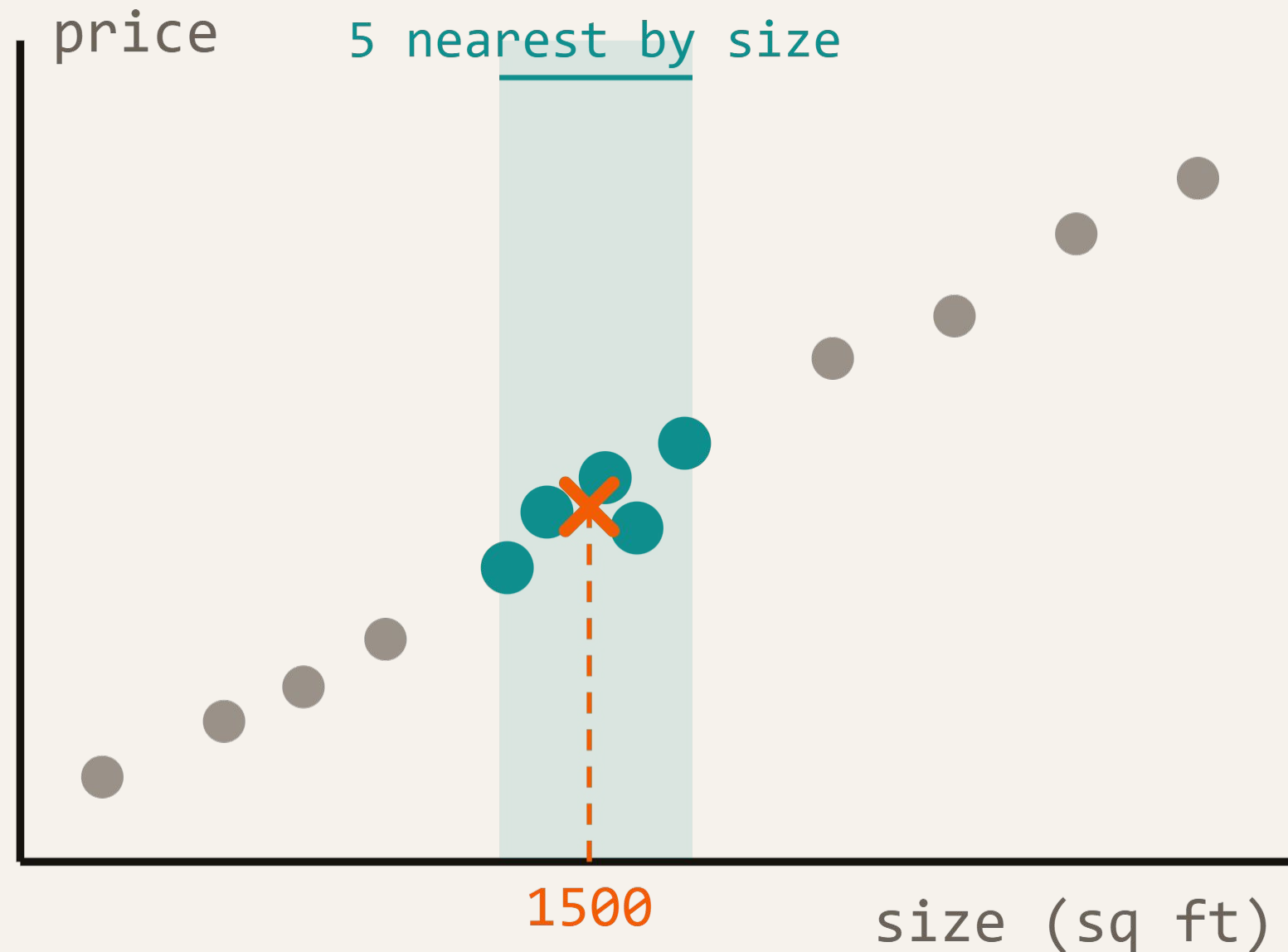
Both formulas work by subtracting values. A column measured in big numbers produces big gaps, a column in small numbers produces small gaps.

```
salary 800000 vs 650000 → gap 150000  
age 34 vs 29 → gap 5
```

Added together, the 5 from age vanishes next to the 150000 from salary. The model is deciding on salary alone.

Scaling rewrites every column onto the same range, so each gets a fair say. Always scale before running KNN.

Same Steps, but the Neighbours Are Averaged



A new house at 1500 sq ft. Nearest means nearest in **size** only, so we measure left and right, never up and down.

Price is the answer we are looking for, so it cannot be used to find the neighbours. Widen the band around 1500 until exactly five houses fall inside it: 1400, 1450, 1520, 1560, 1620.

$$62 + 70 + 75 + 68 + 80 = 355$$

$$355 / 5 = \mathbf{71}$$

Predicted price: 71 lakh. No voting, just the average of the neighbours' prices.

When One Odd House Ruins the Average

ONE NEIGHBOUR SOLD AT 400

62 • 70 • 75 • 68 • 400

average = $675 / 5 = 135$

median = 70

The average has jumped to 135 lakh, far above every ordinary house in the group. The median has barely moved.

The median is the middle value once the numbers are sorted. One extreme number cannot pull the middle of a list.

The average uses every value, so every value can distort it.

Use the average by default. Switch to the median when the data has outliers you cannot remove.

KNN Never Actually Learns Anything

Linear regression ends up with a line. Logistic regression ends up with weights. Naive Bayes ends up with a table of probabilities.

KNN ends up with the training data, unchanged. It simply keeps it and waits.

This is why it is called a lazy learner. All the work is postponed to prediction time.

TRAINING

Instant. Nothing is computed.

EVERY PREDICTION

Slow. Distance to every stored point, every time.

It also means the whole training set has to stay in memory for the model to work at all.

When the Vote Ties, and When the Distance Is Zero

THE VOTE ENDS IN A TIE

With two categories and an odd K this cannot happen. With three or more it can: two neighbours each for two different classes.

$K = 5 \rightarrow A:2 \ B:2 \ C:1$

The sensible rule is to break the tie by distance: whichever tied class has its neighbours closer wins.

Scikit-learn just takes whichever label comes first, so do not rely on it. Keep K odd and the classes uneven.

THE DISTANCE COMES OUT ZERO

Distance is zero only when every feature matches exactly, so the test point sits right on top of a training point.

$$\sqrt{(0^2 + 0^2)} = 0$$

Nothing breaks. Zero is the smallest distance there is, so that point simply comes first in the list of neighbours.

The one exception is distance weighted voting, where a neighbour's weight is 1 divided by its distance. Since $1/0$ is undefined, libraries give the exact match all the weight.

■ THE ONE REAL WEAKNESS

Every prediction measures the distance to every single training point.

$O(n)$

The work grows in direct proportion to the number of training rows.

10 million rows

Ten million distance calculations, for one prediction.

THE FIX

Organise the training points into a tree, so most of them can be skipped without measuring at all.

Next session: KD tree and ball tree, the two structures that make this practical.

The Whole Algorithm in Four Lines

01 Pick K, the number of neighbours to consult.

02 Find the K nearest points, by Euclidean or Manhattan distance.

03 Classification: they vote, majority wins.

04 Regression: average them, or median if there are outliers.

DO NOT FORGET

Scale the features first, or the column with the biggest numbers decides everything.

Keep K odd when there are two categories, so the vote cannot tie.

There is no training phase, which is exactly why prediction is the slow part.